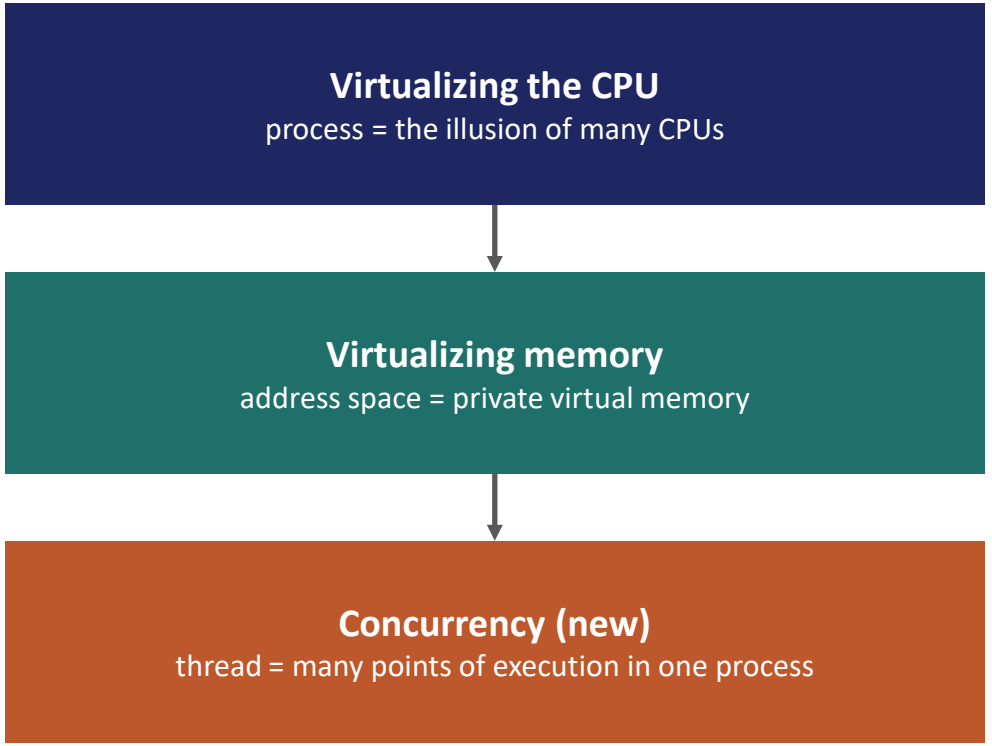
CS3.301: Operating Systems and Networks

Lecture 18 – Introduction to Concurrency

INSTRUCTOR: DR. DEEPAK GANGADHARAN

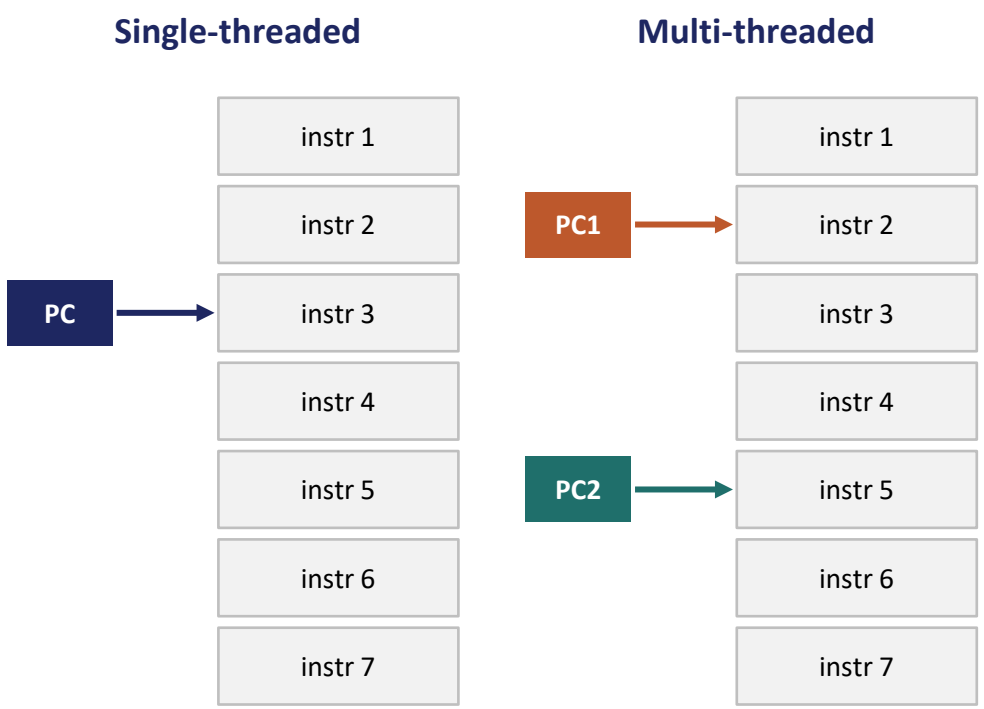
Where We Are: From Virtualization to Concurrency

- So far: one physical CPU became many **virtual CPUs** (processes)
- Memory became a large, private **virtual address space** for every process
- New abstraction inside a single process: the **thread**
- A multi-threaded program has more than one point of execution
- This lecture: why we use threads, and what goes wrong when they share data



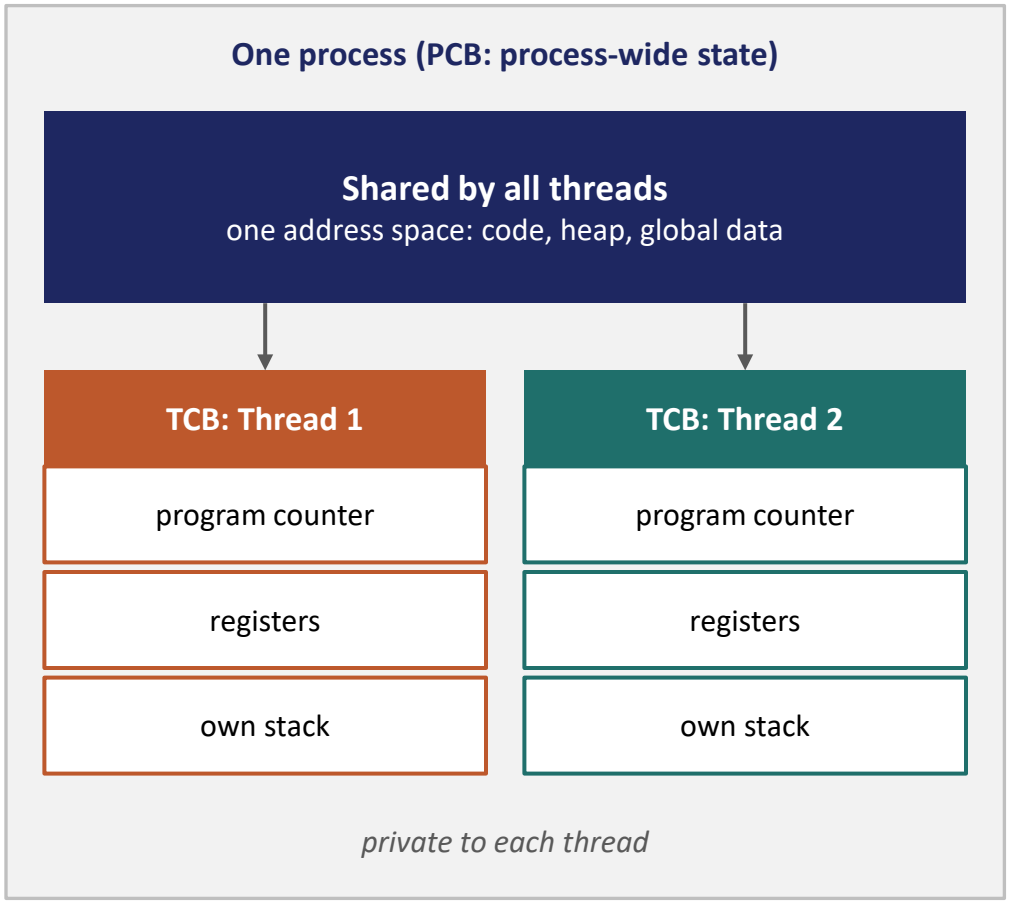
A Thread: Another Point of Execution

- Classic process: **one** point of execution, a single program counter (PC)
- Multi-threaded process: **several** PCs, each fetching and executing its own instructions
- Each thread is like a separate process, with one difference:
  - threads share the same address space
  - so they can access the same data



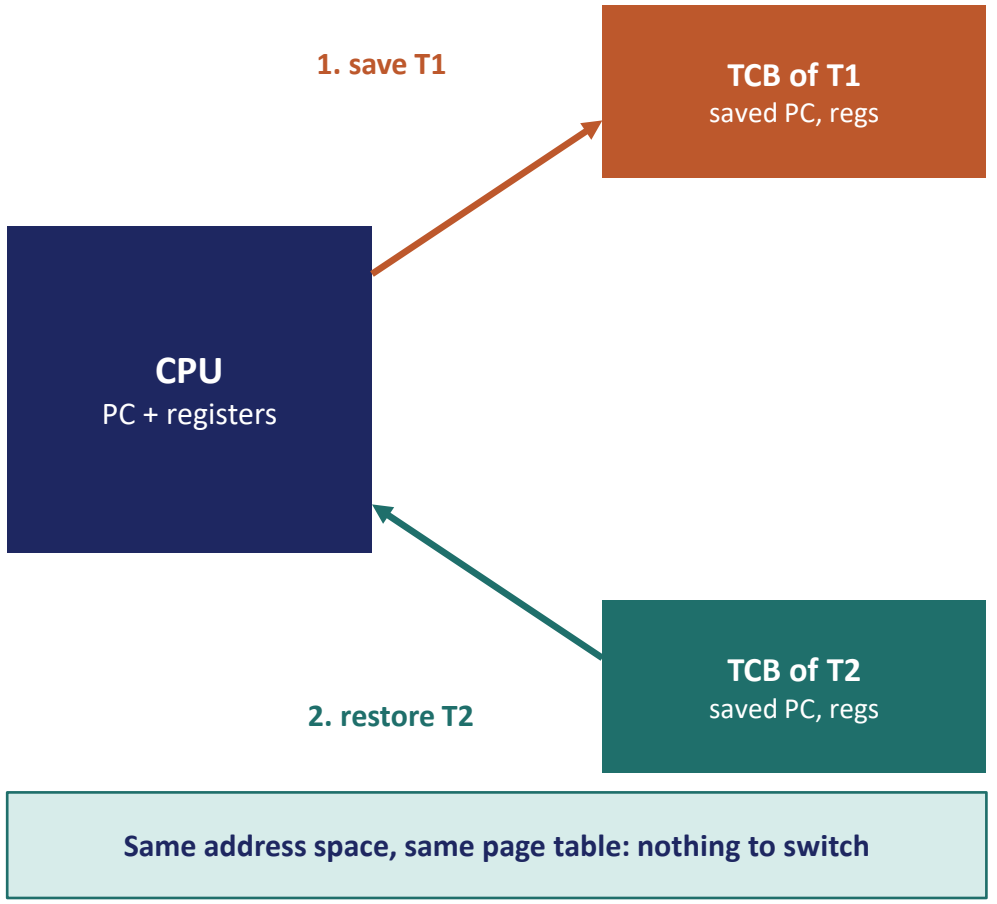
Thread State: PCB vs TCB

- Each thread has its own **program counter** (PC)
- Each thread has its own **private registers** for computation
- Process state lives in a process control block (**PCB**)
- Each thread's state lives in a thread control block (**TCB**): one process can have several



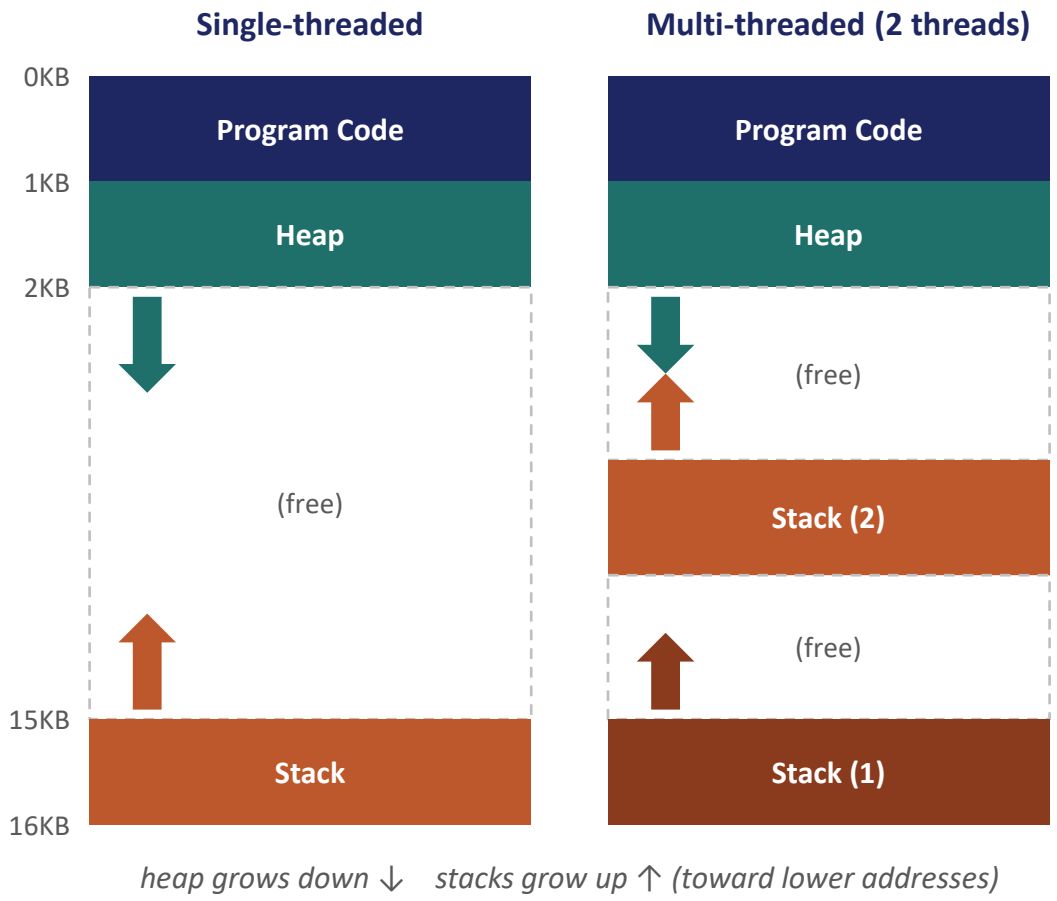
Context Switch Between Threads

- Two threads on one CPU: going from T1 to T2 needs a context switch
- Save T1's register state (PC, eax, ...) into T1's TCB
- Restore T2's register state from T2's TCB, then run T2
- Same idea as a process switch, with one major difference: **the address space stays the same**
- No need to switch which page table is in use



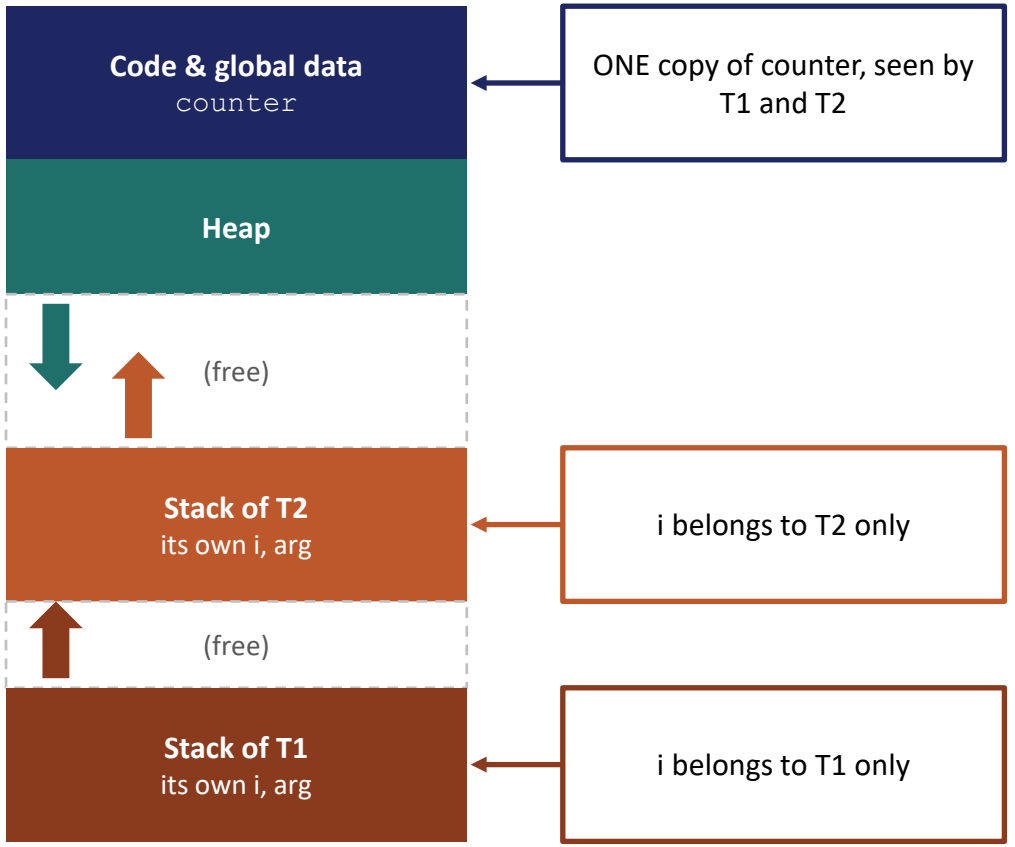
Single-Threaded vs Multi-Threaded Address Space

- Single-threaded: **one stack**, at the bottom of the address space
- Multi-threaded: **one stack per thread**, spread through the address space
- Stack variables, parameters and return values go on the running thread's own stack
- The tidy picture of a heap and stack growing apart is gone, but stacks are usually small



Shared Data vs Thread-Local Storage

- Code, heap and global variables are **shared** by every thread
- Local variables, arguments and return values live on the thread's own stack: **private**
- A write to a global by one thread is visible to all the others
- A local variable is a separate copy in each thread, even when the code is the same function



Why Use Threads? Reason 1: Parallelism

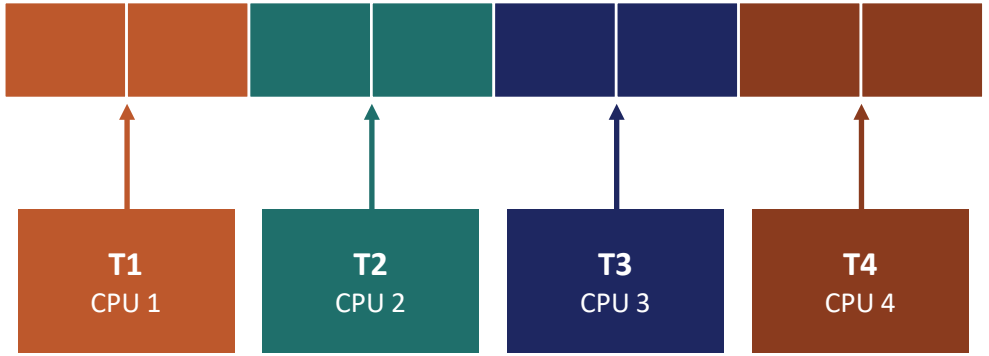
- Example: add two huge arrays, or increment every element by some amount
- One CPU: just do each operation in turn and be done
- Several CPUs: give each CPU a **portion** of the work and run them together
- Turning a single-threaded program into one that does this is called **parallelization**
- A thread per CPU is the natural way to do it

1 thread on 1 CPU



T1 walks through the whole array, element by element

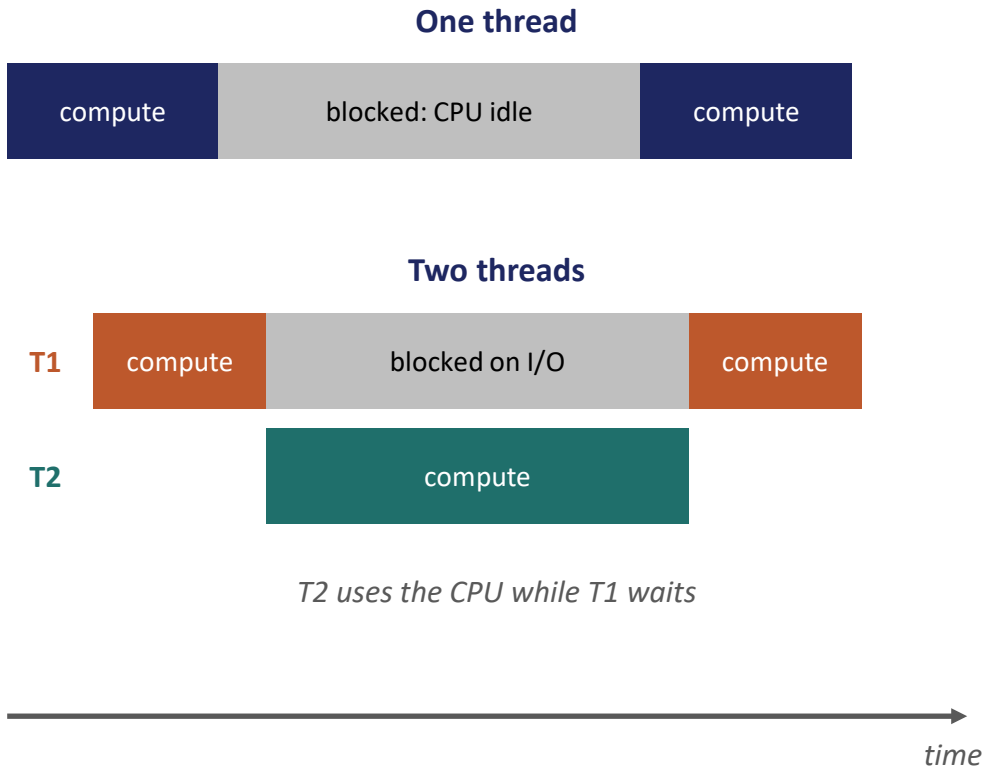
4 threads on 4 CPUs



each thread works on its own portion at the same time

Why Use Threads? Reason 2: Overlapping I/O

- Slow I/O can block a program: waiting to send or receive a message, an explicit disk I/O, even a page fault
- While one thread is blocked, the scheduler can run another thread that is ready
- Overlaps I/O with other activity, like multiprogramming across programs
- Many servers (web servers, database systems) are built this way



Threads vs Processes: Which to Use?

- Both goals above could also be met with multiple processes
- Threads share an address space, so **sharing data is easy**
- Processes are the sounder choice for **logically separate tasks** that need little sharing in memory

	Threads	Processes
Address space	One, shared by all the threads	One private address space per process
Context switch	Registers saved and restored; same page table	Registers saved and restored; page table switched
Sharing data	Easy: every thread sees the same memory	Memory is private to each process
Best suited to	Parallel or I/O-overlapping work on shared data structures	Logically separate tasks, little sharing in memory

Example: Creating Threads

- main creates two threads, T1 and T2, that both run **`mythread()`** with different arguments ("A", "B")
- A new thread may start right away, or wait in the ready state
- **`pthread_join()`** waits for a particular thread to finish
- Three threads in all: main, T1 and T2

```c
#include <stdio.h>
#include <pthread.h>

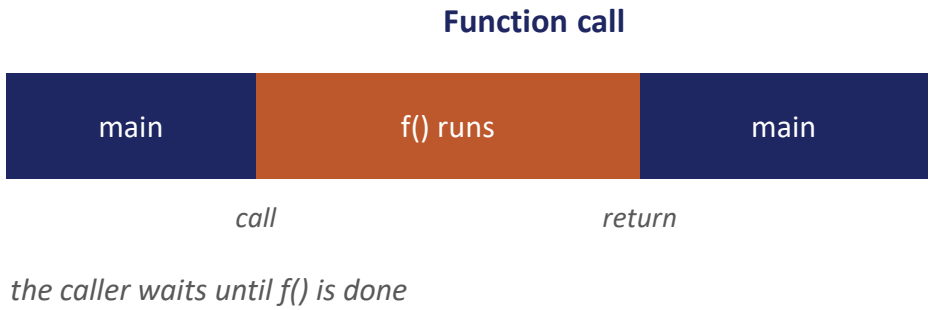
void *mythread(void *arg) {
    printf("%s\n", (char *) arg);
    return NULL;
}

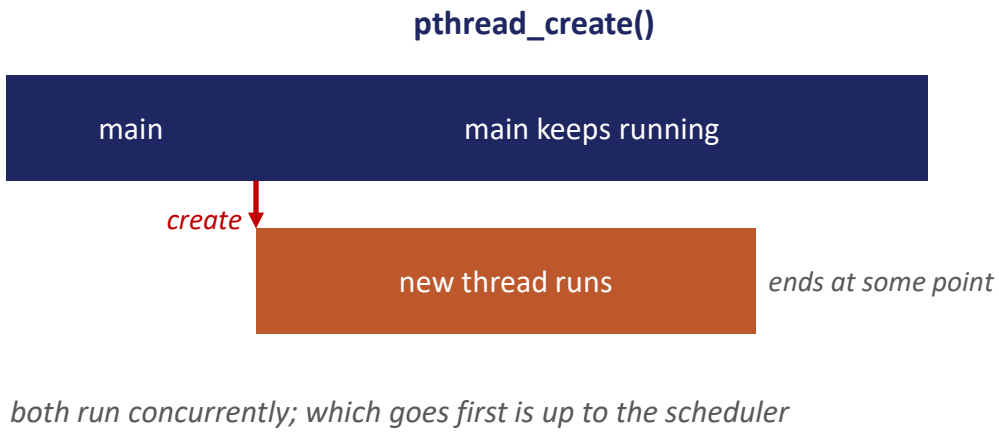
int main(int argc, char *argv[]) {
    pthread_t p1, p2;
    printf("main: begin\n");
    Pthread_create(&p1, NULL, mythread, "A");
    Pthread_create(&p2, NULL, mythread, "B");
    // join waits for the threads to finish
    Pthread_join(p1, NULL);
    Pthread_join(p2, NULL);
    printf("main: end\n");
    return 0;
}
```

Pthread_create / Pthread_join are thin wrappers that exit if the call fails

Thread Creation Is Not a Function Call

- It resembles a function call...
- ...but the system creates a new thread of execution for the routine
- The new thread runs independently of the caller
  - perhaps before create returns, perhaps much later
- The OS scheduler decides what runs next, and it is hard to predict





One Program, Several Possible Orderings

- Same program, three legal executions: the scheduler decides who runs when
- A thread created first is not guaranteed to run first: T2 can print B before T1 prints A

main Thread 1 Thread 2

Trace 1: threads run late	Trace 2: T1 runs at once	Trace 3: T2 before T1
main: prints begin	main: prints begin	main: prints begin
main: creates T1	main: creates T1	main: creates T1
main: creates T2	T1: prints A, returns	main: creates T2
main: waits for T1	main: creates T2	T2: prints B, returns
T1: prints A, returns	T2: prints B, returns	main: waits for T1
main: waits for T2	main: join T1 (already done)	T1: prints A, returns
T2: prints B, returns	main: join T2 (already done)	main: join T2 (already done)
main: prints end	main: prints end	main: prints end
prints: begin, A, B, end	**prints: begin, A, B, end**	**prints: begin, B, A, end**

Sharing Data: The Counter Program

- Two threads each add 1 to a **shared global** counter, 10 million times (1e7)
- Same function for both; the argument ("A" or "B") only labels the output
- Desired final value: **20,000,000**
- **`counter`** is global: one copy shared by both threads; **`i`** is local: one per thread

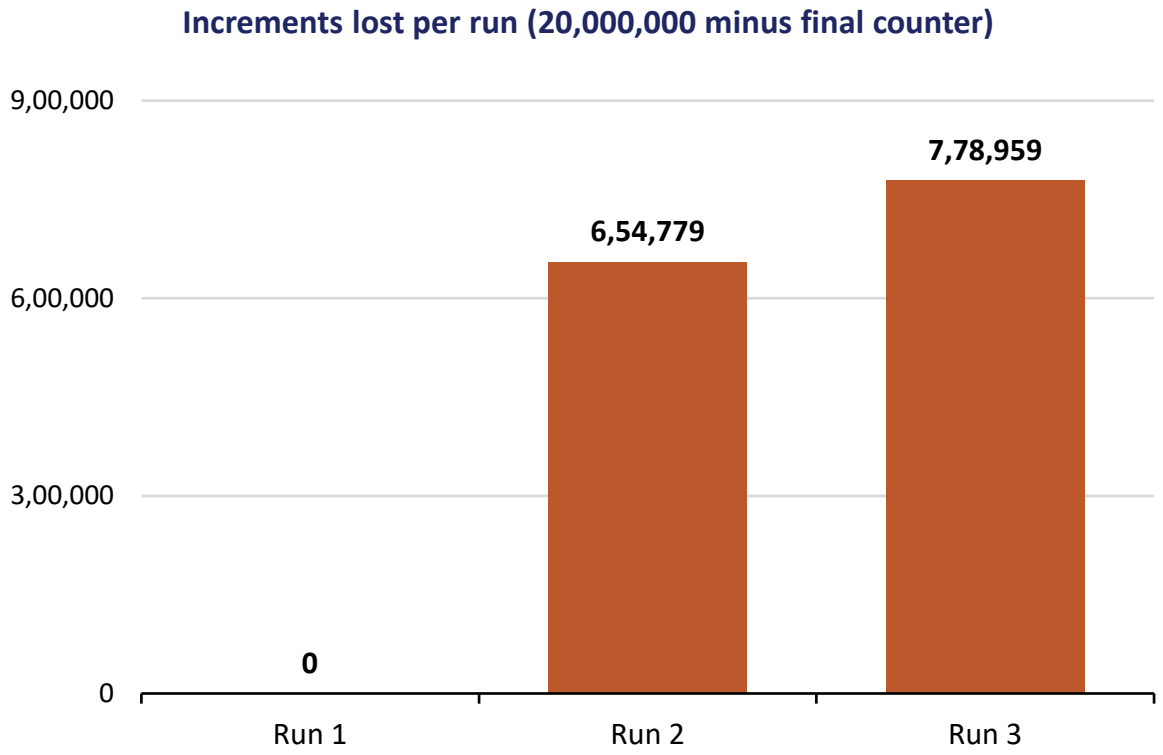
```c
static volatile int counter = 0;

void *mythread(void *arg) {
    printf("%s: begin\n", (char *) arg);
    int i;
    for (i = 0; i < 1e7; i++) {
        counter = counter + 1;
    }
    printf("%s: done\n", (char *) arg);
    return NULL;
}

int main(int argc, char *argv[]) {
    pthread_t p1, p2;
    printf("main: begin (counter = %d)\n", counter);
    Pthread_create(&p1, NULL, mythread, "A");
    Pthread_create(&p2, NULL, mythread, "B");
    Pthread_join(p1, NULL);
    Pthread_join(p2, NULL);
    printf("main: done with both (counter = %d)\n",
           counter);
    return 0;
}
```

Run It: The Answer Changes Every Time

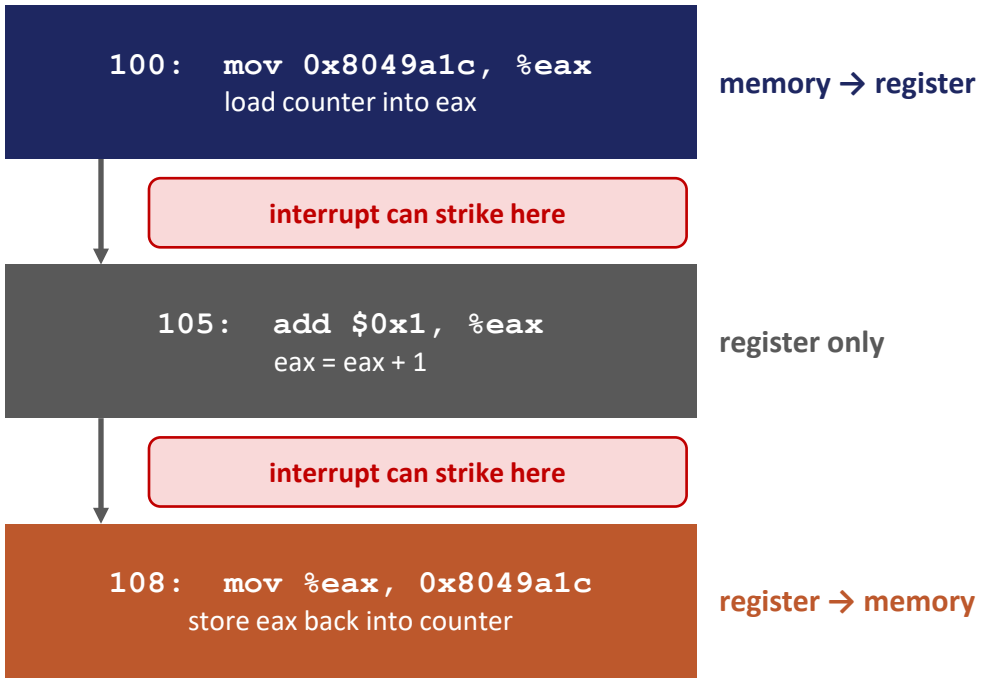
- Sometimes everything works: **20,000,000**
- Other runs print 19,345,221 and 19,221,041
- Wrong, and different each time, even on a single processor
- Computers are supposed to be deterministic. Why does this happen?



Final counter: 20,000,000 | 19,345,221 | 19,221,041

Under the Hood: counter = counter + 1

- One C statement becomes three x86 instructions: load, add, store
- Assume **`counter`** lives at address **`0x8049a1c`**
- A timer interrupt can arrive between any two of these instructions
- Tip: **`objdump -d main`** shows the assembly the compiler generated



Uncontrolled Scheduling: A Detailed Trace

Who	Instruction / event	PC	eax	counter
—	before critical section	100	0	50
T1	mov 8049a1c, %eax	105	50	50
T1	add $0x1, %eax	108	51	50
OS	interrupt: save T1, restore T2	100	0	50
T2	mov 8049a1c, %eax	105	50	50
T2	add $0x1, %eax	108	51	50
T2	mov %eax, 8049a1c	113	51	**51**
OS	interrupt: save T2, restore T1	108	51	51
T1	mov %eax, 8049a1c	113	51	**51**

PC, eax and counter are shown after each instruction

- T1 loads counter (50), adds 1, then an interrupt switches to T2
- T2 runs the whole sequence and stores 51
- T1 resumes with its old eax = 51 and stores 51 again
- **Two increments, one result:** counter is 51, not 52

The Lost Update, Step by Step

- Both threads read the same old value (50) before either one writes
- T2's store of 51 is later overwritten by T1's stale 51
- The update that disappears depends only on where the interrupt lands
- **This is a race condition**: the result depends on timing

Step	T1's eax	counter	T2's eax
start	0	50	0
T1: load counter	**50**	50	0
T1: add 1	**51**	50	0
interrupt → T2: load counter	51	50	**50**
T2: add 1	51	50	**51**
T2: store eax	51	**51**	51
interrupt → T1: store eax	51	**51**	51

Expected 52, got 51: one increment is lost

Key Concurrency Terms

Critical section

Code that accesses a shared resource (usually a variable or data structure) and must not be executed concurrently by more than one thread

Race condition (data race)

Several threads enter the critical section at about the same time and update shared data; the result depends on timing

Indeterminate program

Contains one or more race conditions; its output varies from run to run depending on which threads ran when

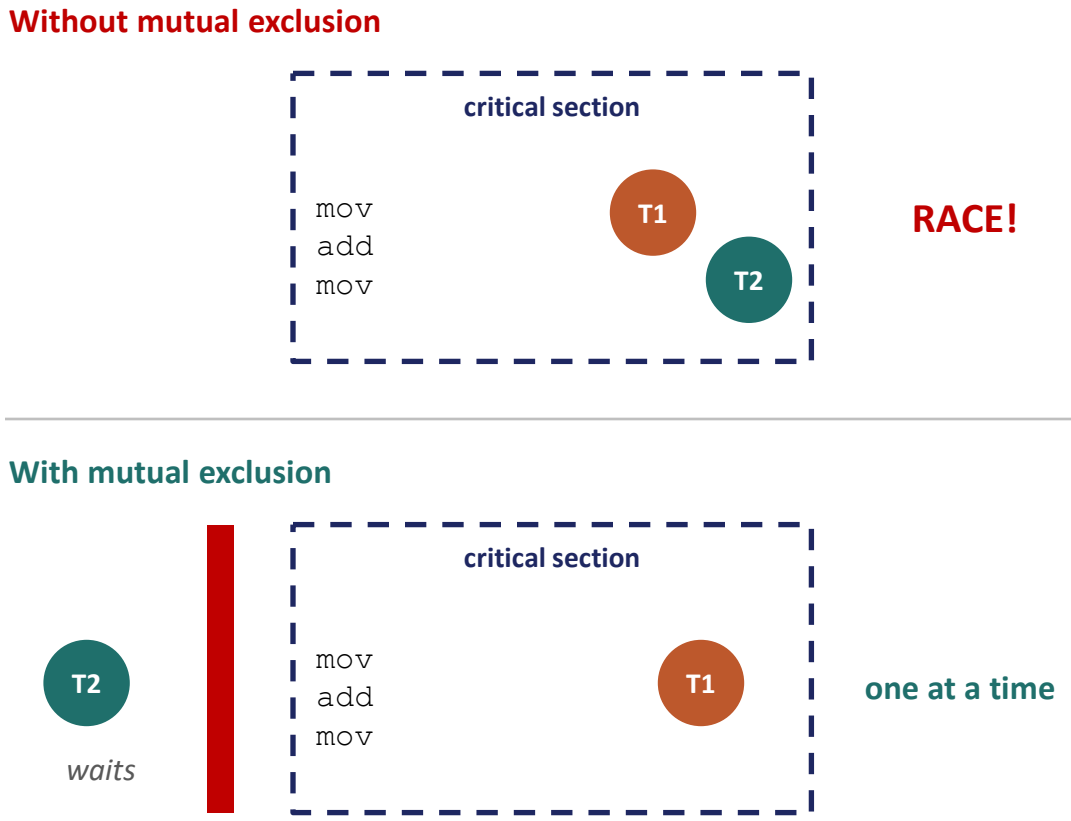
Mutual exclusion

Guarantees that if one thread is executing inside the critical section, the others are prevented from entering it

- Most of this vocabulary comes from Edsger Dijkstra, a pioneer of concurrency and Turing Award winner

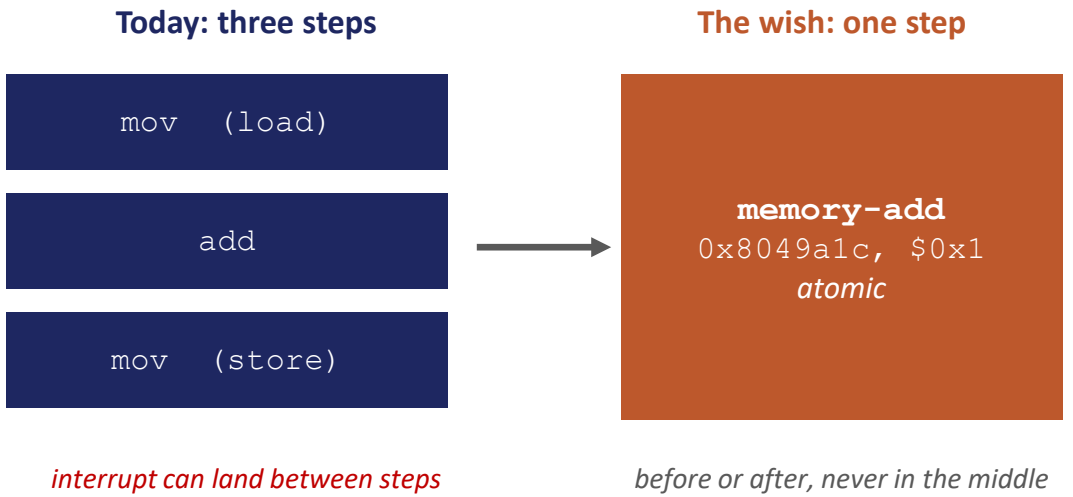
Mutual Exclusion: One Thread at a Time

- The three-instruction update of **`counter`** is the **critical section**
- Without protection, T1 and T2 can both be inside at once: a race
- With mutual exclusion only one thread is inside; the other waits to enter
- Updates never interleave, so the result is correct every time



The Wish for Atomicity

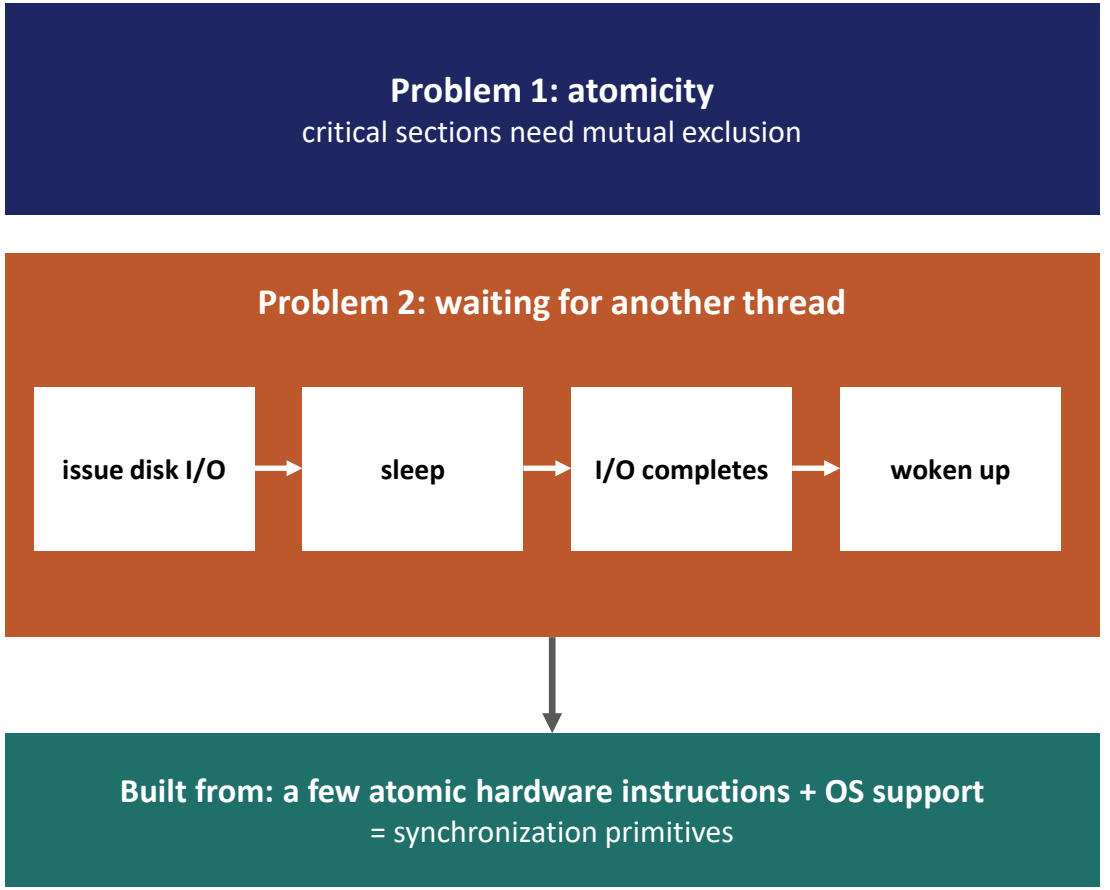
- Wish: one **`memory-add`** instruction that performs the whole update in a single step
- Hardware guarantees an instruction has either not run, or run to completion: an interrupt cannot split it
- **Atomic** = "as a unit" = all or nothing; no in-between state is visible
- But we cannot have an instruction for everything (an atomic B-tree update?)
- Instead: a few useful hardware instructions + OS help = synchronization primitives



Tip: use atomic operations
"All or nothing" is one of the most powerful techniques in systems: architecture, concurrent code, file systems (journaling, copy-on-write), databases (transactions), distributed systems

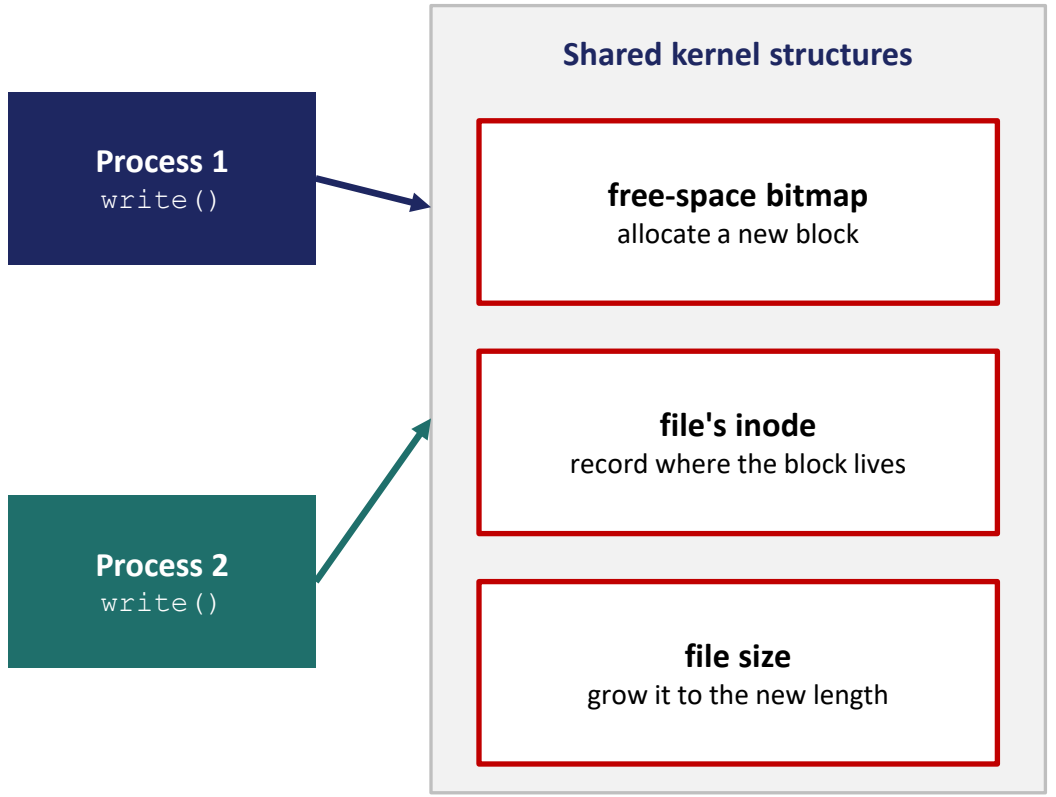
What We Need Next: Synchronization

- Interaction 1: threads update shared data, so we need **atomicity** for critical sections
- Interaction 2: one thread must **wait** for another to act, for example a process sleeping until its disk I/O completes
- Both need support from the hardware and the OS, built correctly and efficiently
- Coming up: mutual exclusion primitives, then condition variables for sleeping and waking



Why Study This in an OS Class?

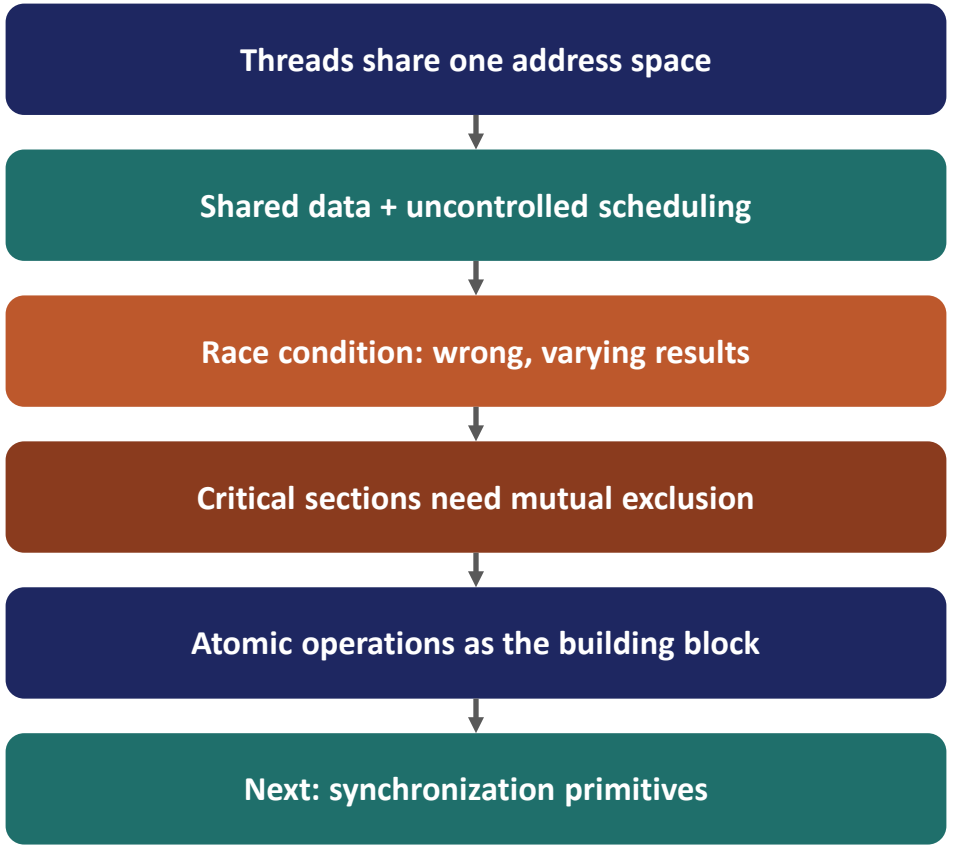
- History: the OS was the first concurrent program; many techniques were invented for it
- Later, multi-threaded processes made application programmers face the same issues
- Example: two processes both write() to append to the same file
  - each must allocate a block, record it in the inode and update the size
  - an interrupt can strike between steps: these are critical sections
- Page tables, process lists, file-system structures and nearly every kernel data structure need proper synchronization



red outlines: updates that must not be interleaved

Summary: Key Takeaways

- A thread is a point of execution with its own PC, registers and stack, inside a shared address space
- Threads give parallelism and overlap of I/O, but make behavior harder to predict
- Scheduling order is not under the program's control
- Unsynchronized updates to shared data cause race conditions and indeterminate results
- Critical sections need mutual exclusion; atomic hardware operations are the foundation
- Practice: the `x86.py` simulator lets you step through thread interleavings



Thank You!!